

user_guide_sidfex

October 28, 2021

1 The SIDFEx user guide

1.0.1 Background

SIDFEx (Sea Ice Drift Forecast Experiment) is a community-effort database which currently (October 2021) comprises more than 168k ice drift forecasts for single buoys, issued by 14 research groups and continuously growing since 2017. The idea originated in ... Since then, the groups regularly submit forecasts which are processed and standardised operationally by a processing chain hosted at DKRZ (German Climate Computation Center/Deutsches Klimarechenzentrum). The data can be downloaded and explored by a software package written in R, called SIDFEx. This user guide allows you to interactively get used to the package and do first analyses with the dataset. It makes no claim to completeness, it is rather meant to give you a first flavour of the software and the dataset. Additionally, there is a PDF version of the user guide which provides a bit more background information and interpretation.

- Author (software packages): Helge Goessling
- Author (this notebook): Valentin Ludwig
- Author (PDF user guide): Helge Goessling
- Contacts: helge.goessling@awi.de; valentin.ludwig@awi.de
- Creation date: 20211027
- Last used on: 20211027
- Purpose: Try SIDFEx user guide

1.0.2 1) Getting started

Important note: This section is only needed for the setup of the package. Once this runs, this section can be skipped.

1.1) Set paths

Prior to starting: Set paths. To this end, do (according to user guide): - create a file `~/SIDFEx` in your home dir - add the line `data.path.fcst = "~/SIDFEx/fcst"` - add the line `data.path.obs = "~/SIDFEx/obs"` - add the lines `indexTable.path.in = "~/SIDFEx/index"`

This can be done by executing the shell script `"setup_sidfex.bash"` which is also provided in this repository. It can be executed by typing `/bin/bash setup_sidfex.bash` in the command line.

Remark: In principle, you can also use different filepaths. This is, however, highly *not* recommended because many functions rely on the filepaths being set as above. If the filepaths differ, the functions

need to be adapted manually, which is both cumbersome and error-prone.

1.2) Installing libraries

Two libraries are required in order to use the SIDFEx database. Their names are `spheRlab` and `SIDFEx`. Both were developed and are maintained by AWI. They are publicly available on GitHub and can therefore be installed using the R package “devtools”, which is part of the conda environment which comes with this repository.

```
[22]: ## Load devtools library (needed to install spheRlab and SIDFEx libraries)
library(devtools)
```

```
[24]: ## Install spheRlab and SIDFEx
devtools::install_github("FESOM/spheRlab") # install spheRlab
devtools::install_github("helgegoessling/SIDFEx") # install SIDFEx
```

Skipping install of 'spheRlab' from a github remote, the SHA1 (29caab8d) has not changed since last install.

Use `force = TRUE` to force installation

Downloading GitHub repo helgegoessling/SIDFEx@master

checking for file '/private/var/folders/d6/cf68qnyn42708tqdllr3pbmjmsjrs2/T/RtmpIGHRbc/remotes14615aba9e27/helgegoessling-SIDFEx-e1e5e9b/DESCRIPTION' ...

preparing 'SIDFEx':

checking DESCRIPTION meta-information ...

checking for LF line-endings in source and make files and shell scripts

checking for empty or unneeded directories

building 'SIDFEx_0.3.1.tar.gz'

Warning: invalid uid value replaced by that for user 'nobody'

Warning: invalid gid value replaced by that for user 'nobody'

1.0.3 2) Obtaining and handling the data

This section describes how to obtain a local copy of the database. Also, it gives pathways for exploring the dataset. First, we need to load the libraries whose installation is described in Section 1.

```
[25]: ## Load libraries
library(spheRlab)
library(SIDFEx)
```

Now, we can download the forecast and observational data. Note that it is important that the filepaths to the `~/SIDFEx/` directory are set as described in Section 1. If you chose a different filepath, you would need to adapt the functions accordingly.

The next cell *downloads* the data, or updates them if new forecasts have been submitted.. If you have already downloaded them previously, there is no need to execute the following cell.

```
[26]: ## Downloading forecast and observational data ###
#res = sidfex.download.fcst() # If you want to download all data, regardless of
↳ whether they have been downloaded already, call the function without
↳ arguments (i.e., use this line)
res = sidfex.download.fcst(comparison.mode = TRUE, from.scratch = FALSE) #
↳ downloads forecast data which have not been downloaded previously (i.e., use
↳ this line).
index = sidfex.load.index() # index is a data frame (smth like a table) which
↳ describes the data. Note that it does not contain the data themselves.
obs = sidfex.download.obs(index = index) # Download observation data described
↳ by index
```

```
[1] "Index download ..."
[1] "Index download done."
[1] "Data download ..."
[1] "Data download done."
[1] "Returning index of files that have been downloaded, and (if not
from.scratch) the more detailed result of sidfex.fcst.search.compareIndexTables"
[1] "Index download ..."
[1] "Index download done."
[1] "Entries only in local index = obsolete files to be deleted: 0"
[1] "Entries only in remote index = new files to be downloaded: 0"
[1] "Identical entries = local files already up-to-date: 168715"
[1] "Entries for same files with differences = files to be downloaded and to
replace local ones: 0"
[1] "To execute these downloads and other changes (if present as indicated
above), resubmit command with comparison.mode=FALSE ."
[1] "Returning index of files that would be downloaded with
comparison.mode=FALSE, and (if not from.scratch) the more detailed result of
sidfex.fcst.search.compareIndexTables"
```

Warning message in sidfex.download.fcst(comparison.mode = TRUE, from.scratch = FALSE):

"Note that, with from.scratch=FALSE, it can not be excluded that the local data differs from the remote data in aspects that are not captured by the index. But that's very unlikely."Warning message in sidfex.targettable.update(download.obs = FALSE, data.path = data.path):

"Make sure that your version of the SIDFEx package is up-to-date, otherwise the manual target table used as

input might be outdated (new targets and/or information on discarded targets might be missing)!"Warning message in sidfex.read.obs(TargetID = tid, data.path = data.path):

"duplicates removed; using only respective last values"Warning message in sidfex.read.obs(TargetID = tid, data.path = data.path):

"duplicates removed; using only respective last values"Warning message in sidfex.read.obs(TargetID = tid, data.path = data.path):

"duplicates removed; using only respective last values"Warning message in sidfex.read.obs(TargetID = tid, data.path = data.path):

```

"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):

```

```

"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):

```

```

"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"duplicates removed; using only respective last values"Warning message in
sidfex.read.obs(TargetID = tid, data.path = data.path):
"data time stamps not always ascending; reordered"Warning message in
sidfex.download.obs(index = index):
"'index' contains TargetIDs not contained in the SIDFEx target table. Consider
updating the target table."

[1] "The following TargetIDs are not downloaded:POLSTERN01"
[1] "Starting download ..."
[1] "Download done."

```

If you only want to load the data from your local machine, use the next cell.

```

[53]: index = sidfex.load.index() # index is a data frame (smth like a table) which
    ↪ describes the data. Note that it does not contain the data themselves.

```

Having downloaded the data and having loaded the index which describes them, we now can dive into exploring the dataset using this index. First, we get an overview over the available systems. “System” is the combination of forecast center and method.

```

[33]: ## Get list of systems (GroupID and MethodID)
systems = paste(index$GroupID, index$MethodID, sep = "_") # Get system list as
    ↪ GroupID_MethodID
systems_unique = unique(systems) # Retrieve unique names
#systems_unique = systems_unique[systems_unique != "awi001_ClimRunVers2017Jul"] #
    ↪ Example to exclude one system (here, awi001_ClimRunVers2017Jul)
print(paste0("There are ", length(systems_unique), " forecast systems in the
    ↪ SIDFEx database."))

```

```

[1] "There are 24 forecast systems in the SIDFEx database."

```

Now that we know how to load the data, we will create a data frame which contains some information about the data.

```

[34]: ## Get data frame with names, nnumber of forecasts, lead times, ensemble sizes
    ↪ and number of time steps
tb1 = as.data.frame(matrix(nrow = length(systems_unique), ncol = 4)) # Create
    ↪ data frame called tb1 with dim 23 (number of systems) by 4 (number of
    ↪ quantities)
rownames(tb1) = systems_unique # use system names as row titles

```

```
colnames(tb1) = c("nFcst", "leadtimerange", "enssize", "nTimeSteps") # use
↳ quantities as column names
```

So far, the data frame only contains the names, but not the data themselves, as is revealed by the next cell.

```
[37]: print(tb1)
```

	nFcst	leadtimerange	enssize	nTimeSteps
awi001_ClimRunVers2017Jul	NA	NA	NA	NA
awi001_ClimRunVers2019May	NA	NA	NA	NA
awi003_iceocean	NA	NA	NA	NA
dmi001_Forecast5d	NA	NA	NA	NA
ecc001_caps	NA	NA	NA	NA
ecc001_giops	NA	NA	NA	NA
ecmw001_SEAS5	NA	NA	NA	NA
esrl001_GFSWinds	NA	NA	NA	NA
esrl001_SeaIceVelocity	NA	NA	NA	NA
metno001_RK2	NA	NA	NA	NA
ncep001_freedrift-ensmean	NA	NA	NA	NA
nrl001_flatearth24	NA	NA	NA	NA
nrl001_gofs3.1-shortrange	NA	NA	NA	NA
nrl001_navyespc-seasonal	NA	NA	NA	NA
nrl001_navyespc-subseasonal	NA	NA	NA	NA
ucl001_IceOceanModelTracer	NA	NA	NA	NA
ukmo001_cplNWP	NA	NA	NA	NA
ukmo001_cplNWP-HR	NA	NA	NA	NA
ukmo001_FOAM	NA	NA	NA	NA
ukmo001v1_cplNWP-HRv1	NA	NA	NA	NA
ukmo001v1_cplNWPv1	NA	NA	NA	NA
ukmo001v1_FOAMv1	NA	NA	NA	NA
uo001_mpasCESM	NA	NA	NA	NA
uw001_IceOceanModel	NA	NA	NA	NA

Now, we want to fill the data frame with actual data.

```
[38]: ## Populate tb1 data frame with data
for (i in 1:length(sysrms_unique)) { # loop over system names
  indices = (sysrms == sysrms_unique[i]) # indices are all indices of the
  ↳ system which is currently handled in the loop
  tb1[i,1] = sum(indices & index$File == index$EnsParentFile) # number of
  ↳ forecasts per system
  tb1[i,2] = mean(index$FcstTime[indices]) # mean forecast time
  tb1[i,3] = mean(index$EnsSize[indices]) # mean ensemble size
  tb1[i,4] = mean(index$nTimeSteps[indices]) # mean time step
}
```

A second look at the now-filled data frame shows a first overview

```
[40]: print(tb1)
```

	nFcst	leadtimerange	enssize	nTimeSteps
awi001_ClimRunVers2017Jul	109	293.097248	10.000000	294.234862
awi001_ClimRunVers2019May	447	216.930624	9.992815	217.961383
awi003_iceocean	142	82.682399	19.964008	83.046224
dmi001_Forecast5d	205	5.000000	1.000000	121.000000
ecccc001_caps	9633	1.998339	1.000000	96.920274
ecccc001_giops	12270	10.000000	1.000000	481.000000
ecmwf001_SEAS5	504	124.000000	50.919489	125.000000
esrl001_GFSWinds	592	10.000000	1.000000	11.000000
esrl001_SeaIceVelocity	12724	10.000000	1.000000	11.000000
metno001_RK2	11472	9.000000	1.000000	9.000000
ncep001_freedrift-ensmean	13090	15.999997	1.049326	16.000000
nrl001_flatearth24	38	120.402314	10.254499	120.902314
nrl001_gofs3.1-shortrange	12716	6.499056	1.000000	6.999056
nrl001_navyespc-seasonal	67	120.264835	14.219780	120.764835
nrl001_navyespc-subseasonal	1871	43.500000	3.964626	44.000000
ucl001_IceOceanModelTracer	79	207.607595	10.000000	208.607595
ukmo001_cp1NWP	7668	10.610329	1.000000	11.610329
ukmo001_cp1NWP-HR	7790	11.000000	1.000000	12.000000
ukmo001_FOAM	7728	8.000000	1.000000	9.000000
ukmo001v1_cp1NWP-HRv1	9367	10.001174	1.000000	11.001174
ukmo001v1_cp1NWPv1	9088	10.001210	1.000000	11.001210
ukmo001v1_FOAMv1	9257	7.001188	1.000000	8.001188
uo001_mpasCESM	26	90.500000	4.438596	363.000000
uw001_IceOceanModel	298	87.048658	4.000000	88.048658

The next step which we will take is to look into the delay, i.e. the time between submission of the forecast and the day that it has been done for.

```
[41]: ## Get quantiles of delay
delay_quantiles = NULL # preallocate delay_quantiles
for (i in 1:length(sysmts_unique)) { # start loop over systems
  delay_quantiles = rbind(delay_quantiles, quantile(index$Delay[sysmts ==
    ↪sysmts_unique[i]])) # get quantiles of delay of current system
}
rownames(delay_quantiles) = sysmts_unique # use system names as row names
```

The output is printed in the next cell. This reveals that some forecasts are submitted in near real time (for example, 75% of the dmi001_Forecast5dforecasts) are submitted within less than a day. Others are submitted later.

```
[42]: print(delay_quantiles)
```

	0%	25%	50%	75%	100%
awi001_ClimRunVers2017Jul	0.241	1.242	64.9600	139.85600	249.057
awi001_ClimRunVers2019May	1.373	166.207	318.8200	710.19600	1531.582
awi003_iceocean	4.348	82.616	138.6320	174.79400	240.843

dmi001_Forecast5d	0.405	0.641	0.8850	0.91900	1.507
eccc001_caps	0.650	0.669	0.8720	2.82900	168.760
eccc001_giops	0.547	0.649	0.8700	3.90200	365.649
ecmwf001_SEAS5	1.917	1.965	3.1450	4.80000	153.658
esrl001_GFSWinds	0.854	0.917	2.1225	135.23675	277.923
esrl001_SeaIceVelocity	0.854	0.938	1.1880	4.14400	366.164
metno001_RK2	2.001	2.043	31.0515	301.38675	1175.397
ncep001_freedrift-ensmean	0.293	59.926	216.7610	430.29700	1655.582
nrl001_flatearth24	7.117	34.137	48.4590	76.46500	115.462
nrl001_gofs3.1-shortrange	0.855	0.860	0.9010	2.16100	297.411
nrl001_navyespc-seasonal	12.120	17.037	40.3660	198.34075	264.341
nrl001_navyespc-subseasonal	1.364	2.370	3.4240	6.29400	199.183
ucl001_IceOceanModelTracer	16.834	69.541	181.6250	242.62400	381.837
ukmo001_cplNWP	1.438	1.627	34.6900	277.66400	575.661
ukmo001_cplNWP-HR	5.245	46.208	201.6630	498.81600	930.661
ukmo001_FOAM	1.437	1.627	28.6910	272.66500	572.661
ukmo001v1_cplNWP-HRv1	0.584	1.335	2.0020	10.83500	36.737
ukmo001v1_cplNWPv1	0.449	0.503	0.5440	0.76100	20.701
ukmo001v1_FOAMv1	0.449	0.503	0.5440	2.62800	20.702
uo001_mpasCESM	37.806	37.807	67.8060	67.80775	97.807
uw001_IceOceanModel	3.003	6.995	10.8240	25.04200	147.980

Now, we want to select a subindex, i.e., a subset of the entire database. For this, the function `sidfex.fcst.search.extractFromTable` is used. Here, we use the forecast from the group `metno001` using the method `RK2` for the target (a.k.a. buoy) `300234063991680`, initialised between day of year 32 and 105.5 in the years 2018 and 2019.

```
[44]: subindex1 = sidfex.fcst.search.extractFromTable(gid = "metno001", mid = "RK2",
→tid = "300234063991680", iy = c(2018,2019), idoy = c(32,105.5)) # get info
→from table (see doc of function for details)
subindex2 = sidfex.fcst.search.extractFromTable(gid = "ecmwf001", mid =
→"SEAS5", tid = "300234063991680", iy = c(2018,2019), idoy = c(32,105.5)) #
→get info from table (see doc of function for details)
```

In the next cell, we define another subindex and then load the corresponding forecast and observation. We use the forecasts done by group `ecmwf001` using the method `SEAS5` for target `300234063991680`, initialised between day of year 32 and 91 in 2019.

```
[50]: print(paste0("Loading data from group ecmwf001 using method SEAS5 for target
→300234063991680, initialised in 2019 between day 32 (", as.Date(32,origin =
→"2018-12-31"), ") and day 91 (", as.Date(91,origin = "2018-12-31"),")"))
subindex3 = sidfex.fcst.search.extractFromTable(gid = "ecmwf001", mid =
→"SEAS5", tid = "300234063991680", iy = c(2019), idoy = c(32,91)) # get info
→from table (see doc of function for details)
fcst = sidfex.read.fcst(files = subindex3, checkfileformat = FALSE) # get all
→ECMWF SEAS5 forecasts for target 300234063991680 between day 32 and day 91
→of 2019. This returns a list with three elements, i.e., 3 forecasts.
→Ensembles of one forecast are merged into one file
```

```
obs = sidfex.read.obs(index = subindex3) # get all observations matching the
↳ criteria of subindex3
```

```
[1] "Loading data from group ecmwf001 using method SEAS5 for target
300234063991680, initialised in 2019 between day 32 (2019-02-01) and day 91
(2019-04-01)"
```

```
Warning message in sidfex.read.obs(index = subindex3):
"duplicates removed; using only respective last values"
```

1.0.4 3) Plotting the data

Now that we know how to load the data and subselect the part which we are actually interested in, we can go over to plotting the data. For this, we are going to plot the ECMWF and the ESRL forecasts. First, we get the relevant time frame.

```
[51]: ## Get time axis between first and last day of forecast/observation
RefYear = fcst$res.list[[1]]$InitYear # Get reference year
RefDayOfYear = fcst$res.list[[1]]$InitDayOfYear # Get reference day of year
nFcst = length(fcst$res.list) # number of forecasts
last.time = sidfex.ydoy2reltime(Year = fcst$res.list[[nFcst]]$LastYear,
↳ DayOfYear = fcst$res.list[[nFcst]]$LastDayOfYear, RefYear = RefYear,
↳ RefDayOfYear = RefDayOfYear) # get time of last data point (only one time)
obs.time = sidfex.ydoy2reltime(Year = obs$data$Year, DayOfYear =
↳ obs$dat$POS_DOY, RefYear = RefYear, RefDayOfYear = RefDayOfYear) # get time
↳ of observation (all timestamps)
##
```

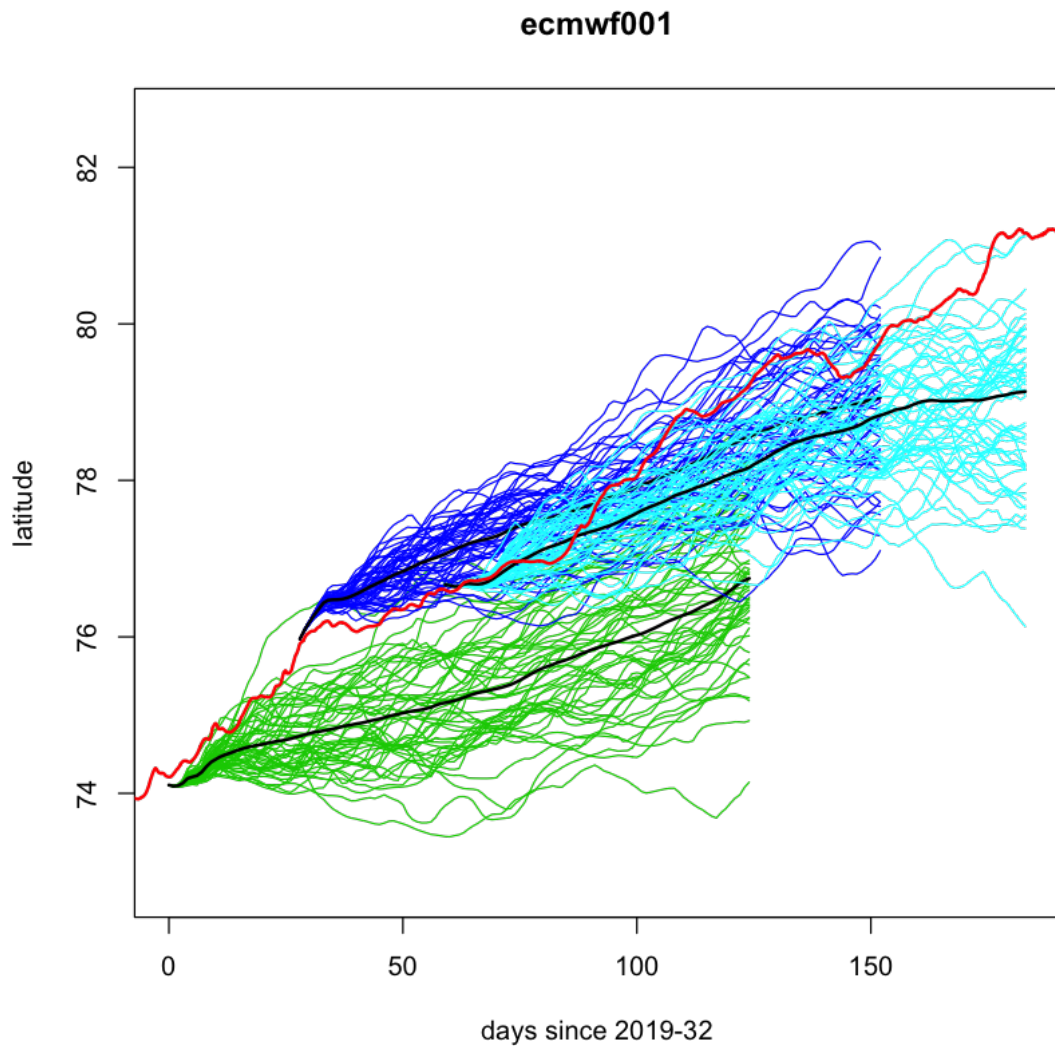
Now, we open a plot in which we then plot the latitude versus the time. There are three ensembles. For all of them, we plot the single trajectories (blue, cyan, green colours) as well as their means (black colour) and the observed trajectory (red color).

```
[52]: # Plot ECMWF forecast #####
## Start the plot
xlim = c(0, last.time) # set limit for x axis
ylim = extendrange(obs$data$Lat[obs.time >= 0 & obs.time <= last.time], f = 0.
↳ 2) # set limit for y axis, expanding the range of the data by 20% (probably
↳ for plotting)
plot(NA, xlim = xlim, ylim = ylim, xlab = paste0("days since ", RefYear, "-",
↳ RefDayOfYear), ylab = "latitude", main = "ecmwf001") # designs the plot
for (i in 1:nFcst) { # loop over forecast ensemble members
  fcst.time = sidfex.ydoy2reltime(Year = fcst$res.list[[i]]$data$Year,
↳ DayOfYear = fcst$res.list[[i]]$data$DayOfYear, RefYear = RefYear,
↳ RefDayOfYear = RefDayOfYear) # get relative time
  LatColumns = which(substr(names(fcst$res.list[[i]]$data), start = 1, stop =
↳ 3) == "Lat") # get columns with latitude. "which" finds those columns whose
↳ first 3 letters are equivalent to "Lat"
```

```

    if (length(LatColumns) > 1) { # if there is more than one latitude column
    ↪ (multiple members), do the following
        for (latcol in LatColumns[2:length(LatColumns)]) { # loop over latitude
    ↪ columns
            lines(fcst.time,fcst$res.list[[i]]$data[,latcol],col = i+2) # plot
    ↪ latitude vs time. col keyword specifies the color
        }
    }
    lines(fcst.time,fcst$res.list[[i]]$data$Lat, col = "black", lwd = 2) #
    ↪ plot ensemble mean
    }
lines(obs.time, obs$data$Lat, col = "red", lwd = 2) # plot observations

```



In the next step, we will plot the ESRL forecast in comparison. The code is pretty much the same, only that it uses the ESRL forecast as input, which differs both in frequency of initialisation and in lead time. Again, we first define a subindex and get the forecast based on this subindex.

```
[54]: ##### Plot ESRL forecast #####
      ## Get time axis between first and last day of forecast/observation

      subindex4 = sidfex.fcst.search.extractFromTable(gid = "esrl001", tid =
        ↪ "300234063991680", iy = c(2019), idoy = c(32,91)) # get info from table (see
        ↪ doc of function for details)

      fcst = sidfex.read.fcst(files = subindex4, checkfileformat = FALSE) # get all
        ↪ ESRL forecasts for target 300234063991680 between day 32 and day 91 of 2019.

      nFcst = length(fcst$res.list) # number of forecasts

      last.time = sidfex.ydoy2reltime(Year = fcst$res.list[[nFcst]]$LastYear,
        ↪ DayOfYear = fcst$res.list[[nFcst]]$LastDayOfYear, RefYear = RefYear,
        ↪ RefDayOfYear = RefDayOfYear) # get time of last data point (only one time)
      ##
```

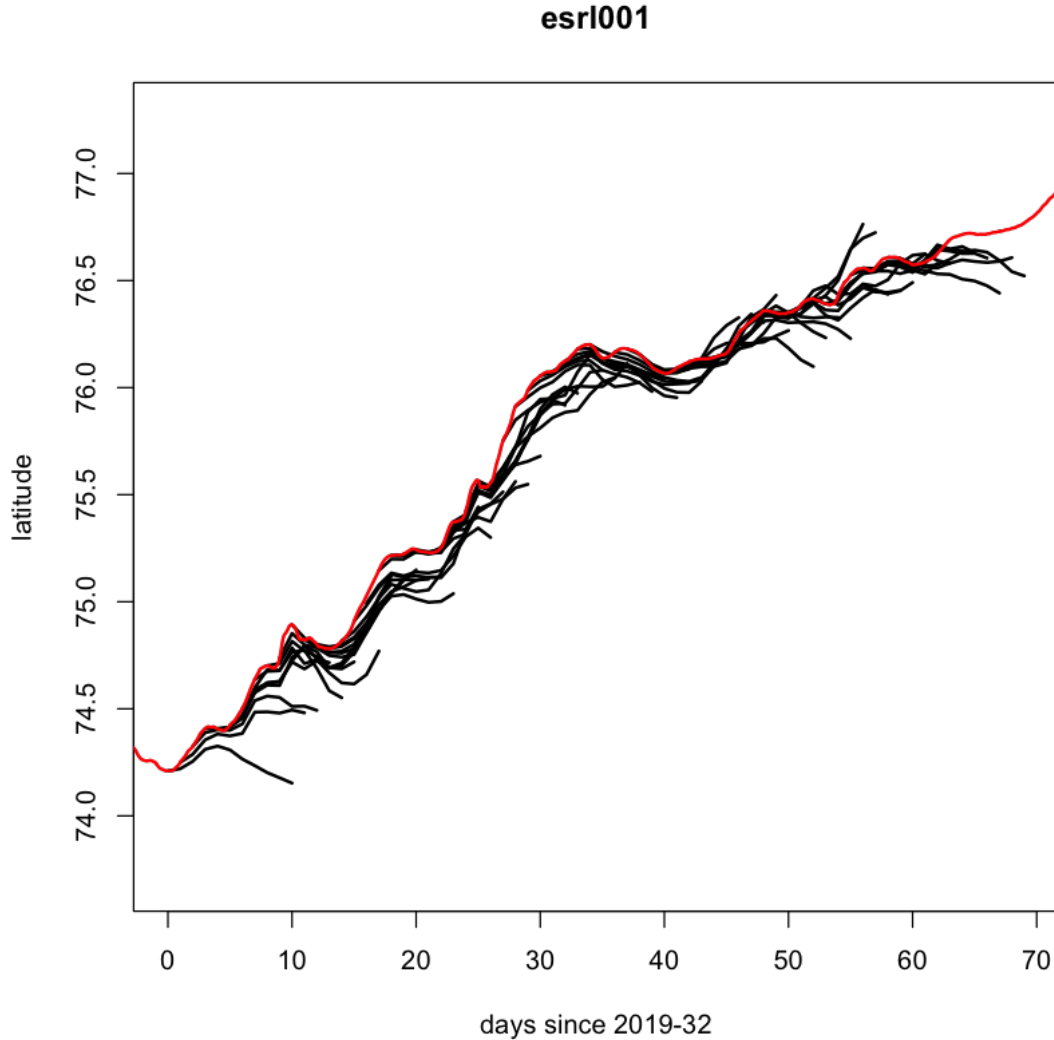
Now, as above, we plot the ESRL forecast latitude versus time, as well as the observed latitude.

```
[55]: ## Start the plot
      xlim = c(0,last.time) # set limit for x axis
      ylim = extendrange(obs$data$Lat[obs.time >= 0 & obs.time <= last.time], f = 0.
        ↪ 2) # set limit for y axis, expanding the range of the data by 20% (probably
        ↪ for plotting)
      plot(NA, xlim = xlim, ylim = ylim, xlab = paste0("days since ", RefYear, "-",
        ↪ RefDayOfYear), ylab = "latitude", main = "esrl001") # designs the plot
      for (i in 1:nFcst) { # loop over forecast ensemble members
        fcst.time = sidfex.ydoy2reltime(Year = fcst$res.list[[i]]$data$Year,
        ↪ DayOfYear = fcst$res.list[[i]]$data$DayOfYear, RefYear = RefYear,
        ↪ RefDayOfYear = RefDayOfYear) # get relative time
        LatColumns = which(substr(names(fcst$res.list[[i]]$data), start = 1, stop =
        ↪ 3) == "Lat") # get columns with latitude. "which" finds those columns whose
        ↪ first 3 letters are equivalent to "Lat"
        if (length(LatColumns) > 1) { # if there is more than one latitude column
        ↪ (multiple members), do the following
          for (latcol in LatColumns[2:length(LatColumns)]) { # loop over latitude
            ↪ columns
            lines(fcst.time,fcst$res.list[[i]]$data[,latcol],col = i+2) # plot
            ↪ latitude vs time. col keyword specifies the color
          }
        }
        lines(fcst.time,fcst$res.list[[i]]$data$Lat, col = "black", lwd = 2) # plot
        ↪ ensemble mean
```

```

}
lines(obs.time, obs$data$Lat, col = "red", lwd = 2) # plot observations
##

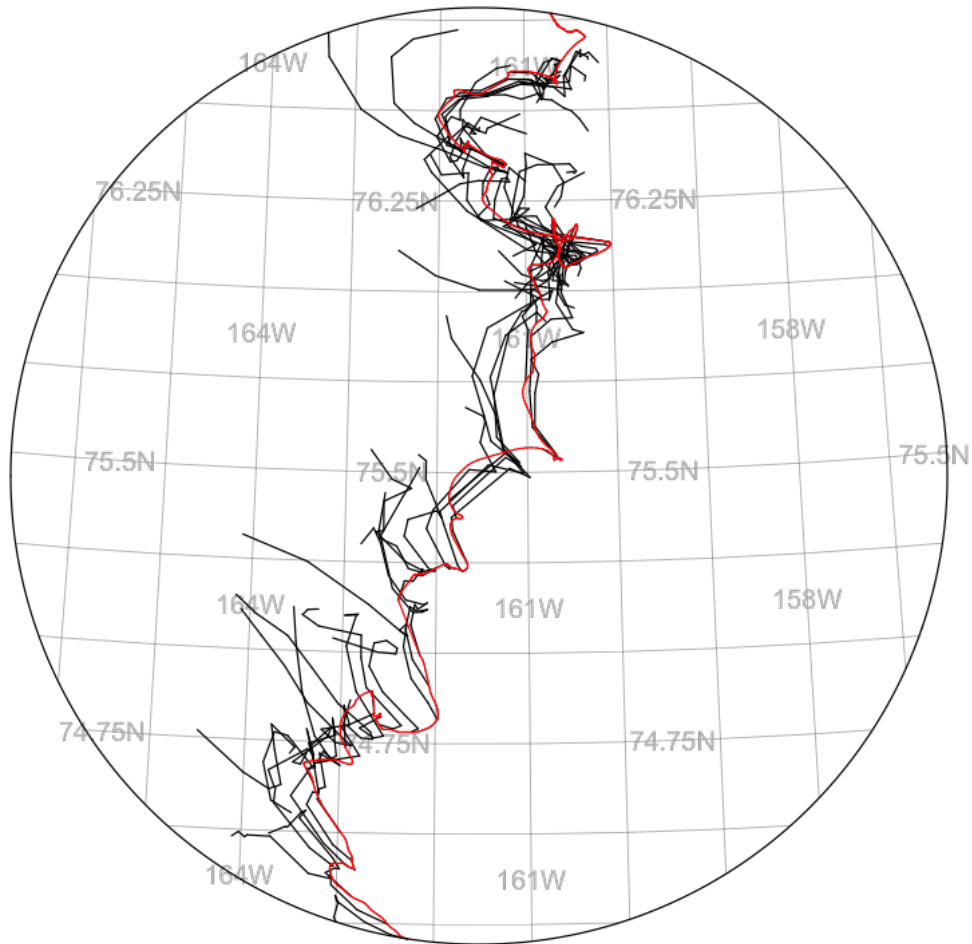
```



We notice that the lead time is shorter (shorter x-axis), but the forecasts are initialised more frequently. In the next step, we will use `sphRlab` for plotting trajectories of the ESRL forecast and of the observed data. Note that, when running this for the very first time, the line `sl.plot.naturalearth...` is needed for downloading the natural earth geoinformation (coastlines etc) if you have not yet downloaded it in previous work. It should be stored in `~/naturalearthdata/`, which should comprise the files: - `ne_10m_coastline.README.html` - `ne_10m_coastline.VERSION.txt` - `ne_10m_coastline.cpg` - `ne_10m_coastline.dbf` - `ne_10m_coastline.prj` - `ne_10m_coastline.rds` - `ne_10m_coastline.shp` - `ne_10m_coastline.shx` - `ne_50m_coastline.README.html` - `ne_50m_coastline.VERSION.txt` - `ne_50m_coast-`

line.cpg - ne_50m_coastline.dbf - ne_50m_coastline.prj - ne_50m_coastline.shp - ne_50m_coastline.shx

```
[58]: ## Now, plot the entire thing as trajectory
which.obs = which(obs.time >= 0 & obs.time <= last.time) # take observations
↳ during forecast time
plot.domain = sl.boundingcircle(lon = obs$data$Lon[which.obs], lat =
↳ obs$data$Lat[which.obs], verbose = FALSE) # define domain for plot
pir = sl.plot.init(projection = "polar", polar.lonlatrot = c(plot.
↳ domain$center_lon, plot.domain$center_lat, 0), polar.latbound = 90-plot.
↳ domain$radius, do.init.device = FALSE) # start plot
## sl.plot.naturalearth(pir, what = "coastline", resolution = "medium") # This
↳ line downloads the coastline file when run for the first time.
↳ Unfortunately, the newest version is not available at this time (see issue
↳ on GitHub here: https://github.com/nvkelso/natural-earth-vector/issues/581,
↳ but an older version (2.0.0) can be downloaded here: https://www.
↳ naturalearthdata.com/downloads/50m-physical-vectors/50m-coastline/ under
↳ "Version history". Temporary solution according to comments on above-
mentioned issue: Download it from https://naturalearth.s3.amazonaws.com/
↳ 50m\_physical/ne\_50m\_coastline.zip, this should be the newest version
sl.plot.lonlatgrid(pir, labels=TRUE) # add grid lines
for (i in 1:nFcst) { # start loop over forecasts
LatColumns = which(substr(names(fcst$res.list[[i]]$data), start = 1, stop = 3)
↳ == "Lat") # extract latitude columns
if (length(LatColumns) > 1) { # loop over all ensemble members if there is more
↳ than one
for (latcol in LatColumns[2:length(LatColumns)]) { # loop over members
sl.plot.lines(pir, lon = fcst$res.list[[i]]$data[, latcol + 1], lat = fcst$res.
↳ list[[i]]$data[, latcol], col = i+2) # plot line (trajectory) for each
↳ member, longitude column selected by latcol + 1 as it is the one after the
↳ latitude column
}
}
sl.plot.lines(pir, lon = fcst$res.list[[i]]$data$Lon, lat = fcst$res.
↳ list[[i]]$data$Lat, col = "black") # plot line if there was only one member
}
sl.plot.lines(pir, lon = obs$data$Lon[which.obs], lat = obs$data$Lat[which.
↳ obs], col = "red") # plot observed trajectory
sl.plot.end(pir, do.close.device = FALSE) # end plotting. If do.close.device ==
↳ FALSE, device will not be closed, i.e. borders of plot will be removed also
↳ for future plots
```



[]:

```
#####
#####
### Section: Remapping and adjusting data in time and space ###
#####
#####
## subindex3.1 = sidfex.fcst.search.extractFromTable(gid = "ecmwf001", tid =
  ↳ "300234063991680", iy = 2019, idoy = c(32)) # take target as above
## fcst = sidfex.read.fcst(files = subindex3.1, checkfileformat = FALSE) # read
  ↳ forecast
## obs.time = sidfex.ydoy2reltime(Year = obs$data$Year, DayOfYear =
  ↳ obs$data$POS_DOY, RefYear = RefYear, RefDayOfYear = RefDayOfYear) # get
  ↳ observation time
## xlim = c(0, 10) # set xlim of plot
```



```

## ylim = extendrange(obs$data$Lat[obs.time >= 0 & obs.time <= 10], f = 0.2) #
  ↳ set ylim of plot
## plot(NA, xlim = xlim, ylim = ylim, xlab = paste0("days since ", RefYear, "-
", RefDayOfYear), ylab = "latitude", main = "ecmwf001")
## #plot(NA, xlim = xlim, ylim = ylim, xlab = paste0("days since ", RefYear, "-
", RefDayOfYear), ylab = "latitude", main = "ecmwf001") # start plot
## fcst.time = sidfex.ydoy2reltime(Year = fcst$res.list[[1]]$data$Year,
  ↳ DayOfYear = fcst$res.list[[1]]$data$DayOfYear, RefYear = RefYear,
  ↳ RefDayOfYear = RefDayOfYear) # get forecast time
## LatColumns = which(substr(names(fcst$res.list[[1]]$data), start = 1, stop =
  ↳ 3) == "Lat") # get latitude columns
## for (latcol in LatColumns[2:length(LatColumns)]) { # loop over ensemble
  ↳ members
  ## lines(fcst.time, fcst$res.list[[1]]$data[, latcol], col = 3) # plot a line
  ↳ for each ensemble member
  ## }
## lines(fcst.time, fcst$res.list[[1]]$data$Lat, col = "black", lwd = 2) # plot
  ↳ mean line
##
## ## Adjust initial location
## fcst.adj.init = sidfex.rot.fcst(obs = obs, fcst = fcst) # adjust initial
  ↳ location
## for (latcol in LatColumns[2:length(LatColumns)]) { # loop over columns
  ↳ (=ensemble members)
  ## lines (fcst.time,fcst.adj.init$res.list[[1]]$data[, latcol], col =
  ↳ "cyan") # plot a line for each adjusted trajectory
  ## }
## lines(fcst.time, fcst.adj.init$res.list[[1]]$data$Lat, col = "blue", lwd =
  ↳ 2) # plot line for adjusted mean trajectory
##
## ## Adjust initial location at DOY 34.5
## fcst.adj.34.5 = sidfex.rot.fcst(obs = obs, fcst = fcst, obsref.Year = 2019,
  ↳ obsref.DayOfYear = 34.5) # adjust at DOY 34.5
## for (latcol in LatColumns[2:length(LatColumns)]) { # loop over columns/
  ↳ ensamble members
  ## lines(fcst.time, fcst.adj.34.5$res.list[[1]]$data[, latcol], col =
  ↳ "orange", lwd = 2) # plot a line for each newly adjusted trajectory
  ## }
## lines(fcst.time, fcst.adj.34.5$res.list[[1]]$data$Lat, col = "brown", lwd =
  ↳ 2) # plot line for adjusted mean trajectory
##
## ## Plot observations
## lines(obs.time, obs$data$Lat, col = "red", lwd = 2) # plot line for
  ↳ observation
##
## ## Remap observations to forecast time axis and plot again

```



```

## obs.remap = sidfex.remaptime.obs2fcst(obs = obs, fcst = fcst) # remap
↳ observations
## obs.remap.time = sidfex.ydoy2reltime(Year = obs.remap$res.
↳ list[[1]]$data$Year, DayOfYear = obs.remap$res.list[[1]]$data$DayOfYear,
↳ RefYear = RefYear, RefDayOfYear = RefDayOfYear) # convert time of remapped
↳ observation to relative time
## lines(obs.remap.time, obs.remap$res.list[[1]]$data$Lat, col = "grey", lwd =
↳ 2) # plot this remapped observation
## #####
##
## subindex3.1 = sidfex.fcst.search.extractFromTable(gid = "ecmwf001" , tid =
↳ "300234063991680", iy = 2019, idoy = c(32)) # again, get target as subindex
## fcst = sidfex.read.fcst(files = subindex3.1, checkfileformat = FALSE) # and
↳ read forecast corresponding to target
## obs.time = sidfex.ydoy2reltime(Year = obs$data$Year, DayOfYear =
↳ obs$data$POS_DOY, RefYear = RefYear, RefDayOfYear = RefDayOfYear) # and
↳ convert day of year to relative time
## xlim = c(0,10) # set x limit for plot
## ylim = extendrange(obs$data$Lat[obs.time >= 0 & obs.time <= 10], f=0.2) #
↳ set y limit for plot
## plot(NA, xlim = xlim, ylim = ylim, xlab = paste0("days since ", RefYear, "-
", RefDayOfYear), ylab = "latitude", main = "ecmwf001") # open plot
## fcst.time = sidfex.ydoy2reltime(Year = fcst$res.list[[1]]$data$Year,
↳ DayOfYear = fcst$res.list[[1]]$data$DayOfYear, RefYear = RefYear,
↳ RefDayOfYear = RefDayOfYear) # convert time of forecast to relative time
## LatColumns = which(substr(names(fcst$res.list[[1]]$data), start = 1, stop =
↳ 3) == "Lat") # get latitude columns
## for (latcol in LatColumns[2:length(LatColumns)]) { # loop over latitude
↳ columns
    ## lines(fcst.time, fcst$res.list[[1]]$data[,latcol], col = 3) # plot
↳ latitude of each member
    ## }
## lines(fcst.time, fcst$res.list[[1]]$data$Lat, col = "black", lwd = 2) # plot
↳ mean latitude
##
## # now with adjustment of the initial location (default of sidfex.rot.fcst())
## fcst.adj.init = sidfex.rot.fcst(obs = obs, fcst = fcst) # adjust initial
↳ location
## for (latcol in LatColumns[2:length(LatColumns)]) { # loop over latitude
↳ columns
    ## lines(fcst.time, fcst.adj.init$res.list[[1]]$data[,latcol], col =
↳ "cyan") # plot latitude for each member
    ## }
## lines(fcst.time, fcst.adj.init$res.list[[1]]$data$Lat, col = "blue", lwd =
↳ 2) # plot mean latitude
##

```

```

## # now with adjustment of the location at DOY=34.5
## # Comments see sections above
## fcst.adj.34.5 = sidfex.rot.fcst(obs = obs, fcst = fcst, obsref.Year = 2019,
  ↳obsref.DayOfYear = 34.5)
## for (latcol in LatColumns[2:length(LatColumns)]) {
  ## lines(fcst.time, fcst.adj.34.5$res.list[[1]]$data[,latcol], col =
  ↳"orange")
  ## }
## lines(fcst.time, fcst.adj.34.5$res.list[[1]]$data$Lat, col = "brown", lwd =
  ↳2)
##
## # plot the observations
## lines(obs.time, obs$data$Lat, col = "red", lwd = 2)
##
## # now remap observations to fcst time axis and re-plot
## obs.remap = sidfex.remapttime.obs2fcst(obs = obs, fcst = fcst) # remap time
  ↳of observation to time of forecast
## obs.remap.time = sidfex.ydoy2reltime(Year = obs.remap$res.
  ↳list[[1]]$data$Year, DayOfYear = obs.remap$res.list[[1]]$data$DayOfYear,
  ↳RefYear = RefYear, RefDayOfYear = RefDayOfYear) # convert this time to
  ↳relative time
## lines(obs.remap.time, obs.remap$res.list[[1]]$data$Lat, col = "grey", lwd =
  ↳2) # plot observation with remapped time

```

[]:

```

#####
#####
### Section: Quantitative forecast evaluation ###
#####
#####

## Speedangle plot
## errs = sidfex.evaluate(fcst = fcst.adj.init) # evaluate forecast which was
  ↳adjusted to initial date
## colbar = sidfex.plot.speedangle(read.fcst.res = fcst.adj.init, points.type =
  ↳"b", col.by = "DaysLeadTime", device = NULL) # make speedangle plot

## More quantitive evaluation
## subind.allmethods = sidfex.fcst.search.extractFromTable(tid =
  ↳"300234061872720", iy = c(2018,2019), idoy = c(335,74), InheritFromParent =
  ↳TRUE) # get all forecasts for all systems for target tid between 2018, day
  ↳335 and 2019, day 74
## systms = paste(subind.allmethods$GroupID,subind.allmethods$MethodID, sep =
  ↳"_") # get list with strings describing the systems
## systms_unique = unique(systms) # get list with duplicates removed from systms
## tb1 = as.data.frame(matrix(nrow = length(systms_unique), ncol = 4)) #
  ↳intialise data frame

```

```

## rownames(tb1) = systms_unique # use system names as row names
## colnames(tb1) = c("nFcst", "leadtimerange", "enssize", "nTimeSteps") # use
  ↳ string for variable as column names
## for (i in 1:length(systms_unique)) { # loop over system names
  ## indices = (systms == systms_unique) # get boolean vector of system
  ## tb1[i,1] = sum(indices) # get number of forecasts (sum of boolean
  ↳ values)
  ## tb1[i,2] = mean(subind.allmethods$FcstTime[indices]) # get mean
  ↳ forecast time
  ## tb1[i,3] = mean(subind.allmethods$EnsSize[indices]) # get mean
  ↳ ensemble size
  ## tb1[i,4] = mean(subind.allmethods$nTimeSteps[indices]) # get mean
  ↳ number of time steps
  ## }

## ##
systms_selected = systms_unique[(tb1$nFcst/tb1$enssize > 10)] # select those
  ↳ systems which have more than 10 ensembles in the considered period
## Remove two systems which are not in the user guide example
systms_selected = systms_selected[which(systms_selected !=
  ↳ "awi001_ClimRunVers2019May")] # remove awi001_ClimRunVers2019May from list
  ↳ of selected systems
systms_selected = systms_selected[which(systms_selected != "ecmwf001_SEAS5")]#
  ↳ remove ecmwf001_SEAS5 from list of selected systems
## Nmethods = length(systms_selected) # number of so-selected systems
## remap.time = seq(0, 10) # sequence of numbers from 0 to 10, including the end
## eval.results = list() # preallocate list with results
## for (i in 1:Nmethods) { # loop over systems
  ## subind = subind.allmethods[systms %in% systms_selected[i], ] # get
  ↳ all forecasts for system currently looped over
  ## fcst.orig = sidfex.read.fcst(files = subind, checkfileformat =
  ↳ FALSE, verbose = FALSE) # read original forecasts
  ## fcst.remap = sidfex.remaptime.fcst(fcst = fcst.orig, newtime.
  ↳ DaysLeadTime = remap.time, verbose = FALSE) # remap to new time axis
  ## fcst.adjust = sidfex.rot.fcst(fcst = fcst.remap) # rotate forecast
  ## eval.results[[i]] = sidfex.evaluate(fcst = fcst.adjust, verbose =
  ↳ FALSE)$multifcst.stats # evaluate forecast
  ## }

## plotdir = "~/plots/user_guide/" # define filepath where plot shall be saved
## pdf(paste0(plotdir,"multi_fcst_mean_error.pdf") ) # open file where plot
  ↳ shall be saved
## plot(NA, xlim = range(remap.time), ylim = c(0,40), main = "multi-forecast-
  ↳ mean error of (ensemble-mean) position", xlab = "lead time (days)", ylab =
  ↳ "great-circle distance (km)") # design plot
## for (i in 1:Nmethods) { # start to loop over systems
  ## print(systms_selected[[i]]) # print currently used system

```

```

    ## dat = eval.results[[i]]$ens.mean.gc.dist/1000 # get ensemble mean
    ↪ distance
    ## lines(remap.time, dat$mean + 2*dat$st.err, col = i, lty = 2, lwd =
    ↪ 2) # plot upper bound (mean + 2sigma)
    ## lines(remap.time, pmin(dat$mean - 2*dat$st.err), col = i, lty = 2,
    ↪ lwd = 2) # plot lower bound (mean - 2 sigma)
    ## lines(remap.time, dat$mean, col = i, lwd = 2) # plot mean
    ## }
## legend("topleft", legend = c(sysrms_selected, "95% confidence"), lty =
    ↪ c(rep(1,Nmethods), 2), bty = "n", col = c(1:Nmethods, 8), lwd = 2) # add
    ↪ legend
## dev.off() # turn off device
pdf(paste0(plotdir,"error_vs_spread.pdf") ) # open file for saving next plot
plot(NA, xlim = range(remap.time), ylim = c(0,40), main = "error versus spread
    ↪ (nrl001_navyespc-subseasonal)", xlab = "lead time (days)", ylab = "great-
circle distance (km)") # design plot
i = which(sysrms_selected == "nrl001_navyespc-subseasonal") # use only
    ↪ nrl001_navyespc-subseasonal
dat = eval.results[[i]]$ens.mean.gc.dist/1000 # get mean distance
lines(remap.time, dat$mean + 2*dat$st.err, col = "cyan", lty = 2, lwd = 2) #
    ↪ plot plus 2 sigma line
lines(remap.time, dat$mean - 2*dat$st.err, col = "cyan", lty = 2, lwd = 2) #
    ↪ plot minus 2 sigma line
lines(remap.time, dat$mean, col = "cyan", lwd = 2) # plot mean

dat = eval.results[[i]]$ens.spread.gc.dist/1000 # get spread
lines(remap.time, dat$mean + 2*dat$st.err, col = "orange", lty = 2, lwd = 2) #
    ↪ plot plus 2 sigma line
lines(remap.time, dat$mean - 2*dat$st.err, col = "orange", lty = 2, lwd = 2) #
    ↪ plot minus 2 sigma line
lines(remap.time, dat$mean, col = "orange", lwd = 2) # plot mean
legend("topleft", legend = c("mean error of ensemble mean", "mean ensemble
    ↪ spread", "95% confidence"), col = c("cyan", "orange", "grey"), bty = "n",
    ↪ lty = c(1,1,2), lwd = 2) # add legend
dev.off() # turn off device

```